DAA Practical 5

8-Queens Problem Using Backtracking

Top 60 Viva Questions & Answers

Core concepts & quick formulas

Board: N × N matrix; 1 = Queen, 0 = empty cell.
Safety rules: no shared row, column, or diagonal.
Search: place one Queen per row; try each column; undo a placement if the next row cannot be solved.
Base case: row == N means all N Queens have been placed successfully.
Time estimate: commonly O(N·N!) for this row-by-row search with O(N) safety checks.
Space: $O(N^2)$ for the board plus O(N) recursion stack; total $O(N^2)$.

First solution produced by this exact code

With the first Queen fixed at (0,0) and columns tested from left to right, the Queen columns by row are: **0, 4, 7, 5, 2, 6, 1, 3**.

Important distinction: recursion vs backtracking

Recursion is a function calling itself. Backtracking is the strategy of trying a choice, exploring it, and undoing it if it fails. This program uses recursion to implement backtracking.

A. Problem, board and core concepts

Q1. What is the N-Queens problem?

Place N Queens on an N×N chessboard so that no two Queens attack each other. This means no two share a row, column, or diagonal.

Q2. What is the objective of this practical?

Place eight Queens on an 8×8 board, with the first Queen fixed at (0,0), and use backtracking to place the remaining seven safely.

Q3. How does a Queen move in chess?

A Queen can move any number of squares horizontally, vertically, or diagonally, provided no piece blocks its path.

Q4. What does a value 1 represent in the board matrix?

A 1 represents a Queen placed in that cell. A 0 represents an empty cell.

Q5. Why is the board represented by a two-dimensional array?

A chessboard has rows and columns, so board[row][col] naturally represents each cell. The 8×8 array stores all 64 positions.

Q6. What does #define N 8 do?

It defines N as 8, which is used as the board dimension and as the upper limit for row and column loops.

Q7. What does int board[N][N] = {0} do?

It creates an N×N integer array and initializes every element to zero, meaning the board starts empty.

Q8. What does board[0][0] = 1 mean?

It places the first Queen in the first row and first column. C++ array indexing starts at zero.

Q9. Why does solveNQueens start with row 1?

Because row 0 already contains the fixed Queen at column 0. The algorithm therefore places the remaining Queens starting from the second row.

Q10. What is backtracking?

Backtracking builds a solution one choice at a time. If a choice leads to a dead end, it undoes that choice and tries another option.

Q11. What are the main stages of backtracking in this program?

Choose a column, check safety, place a Queen, recursively solve the next row, and remove the Queen if that path fails.

Q12. Why is backtracking suitable for N-Queens?

The problem has constraints that must all be satisfied. Backtracking checks constraints during construction and abandons invalid partial arrangements early.

B. Safety checks and recursive search

Q13. What is the purpose of isSafe(row, col)?

It checks whether placing a Queen at the given cell would conflict with any Queen already placed in an earlier row.

Q14. What does the Boolean return type of isSafe mean?

It returns true when the position is safe and false when an existing Queen attacks that position.

Q15. Why does isSafe check the same column?

Two Queens in the same column can attack each other. The function scans earlier rows in the candidate column to detect this conflict.

Q16. Why does the column-check loop start at row zero and stop before row?

Only rows above the current row can contain previously placed Queens because the algorithm fills the board from top to bottom.

Q17. How does the upper-left diagonal check work?

It starts at (row–1, col–1) and decreases both row and column indices, moving diagonally up and left until it reaches a board boundary.

Q18. How does the upper-right diagonal check work?

It starts at (row–1, col+1), decreases the row index and increases the column index, moving diagonally up and right until a boundary is reached.

Q19. Why are only upper diagonals checked?

All previously placed Queens are in earlier rows. Any Queen below the current row has not been placed yet, so only upward diagonals can contain an existing Queen.

Q20. Why does isSafe not check the same row?

The algorithm places exactly one Queen per row. It selects one position in a row and moves to the next row, so no other Queen has been placed in that row.

Q21. What is the base case in solveNQueens?

if (row == N) return true. Reaching row 8 means a Queen has been successfully placed in each of the eight rows.

Q22. Why is a base case necessary in recursion?

It provides a stopping condition. Without it, recursive calls would continue without a valid termination condition.

Q23. Why does the program try every column from 0 to N−1?

The safe column is not known in advance. Trying each column explores the possible placements for the current row.

Q24. What happens when isSafe returns false?

The program skips that column and continues the loop to test the next column.

Q25. What happens when a safe position is found?

The program sets board[row][col] to 1 and recursively calls solveNQueens(row+1) to place a Queen in the next row.

Q26. What does solveNQueens(row + 1) do?

It makes a recursive call for the next row, continuing the search until all Queens are placed or a row has no valid position.

Q27. What is the purpose of board[row][col] = 0?

It removes a Queen placed during an unsuccessful attempt. This restores the board so another column can be tried; this is the backtracking step.

Q28. What would happen if the reset statement were removed?

Failed placements would remain on the board and incorrectly block later choices. The algorithm would no longer explore alternatives correctly.

Q29. Why does the function return true immediately when a recursive call succeeds?

A complete valid arrangement has been found. Returning true propagates success through the recursive calls and stops further searching.

Q30. Does this program find all possible solutions?

No. It returns as soon as it finds the first valid solution. To find all solutions, it must record each complete board and continue searching instead of stopping at the first success.

Q31. Why does the program print the board only after solveNQueens succeeds?

The matrix is printed only when a complete valid arrangement has been found. Otherwise it prints that no solution exists.

C. Backtracking theory and complexity

Q32. What is the first solution produced by this exact column-order search?

With the fixed Queen at (0,0) and columns tried from 0 upward, the row-wise Queen columns are 0, 4, 7, 5, 2, 6, 1, 3.

Q33. How can you verify that the final matrix is valid?

Check that it contains eight 1s, exactly one 1 in each row and column, and no pair of Queens shares either diagonal.

Q34. What is recursion?

Recursion is a technique in which a function calls itself to solve a smaller or next part of a problem, until a base case is reached.

Q35. What is the difference between recursion and backtracking?

Recursion is a function-calling technique. Backtracking is a search strategy that often uses recursion to explore choices and undo choices that fail.

Q36. What is a state-space tree?

It is a conceptual tree of possible choices. Each level represents a row, and each branch represents a candidate column for that row.

Q37. What is pruning in backtracking?

Pruning means stopping exploration of a partial solution as soon as it violates a constraint or cannot lead to a valid solution.

Q38. What is a promising node?

A node is promising if the partial placement does not violate the constraints checked so far and may still lead to a complete solution.

Q39. What is the time complexity of the N-Queens backtracking algorithm?

A commonly quoted upper-bound estimate for row-by-row placement with distinct columns is $O(N \cdot N!)$. Each safety check may scan up to O(N) cells, and at most N! column permutations are explored. Actual work is reduced by diagonal pruning.

Q40. Why is N! involved in the complexity estimate?

There is one Queen per row and no column can be reused. The number of possible assignments of distinct columns across N rows is N!, before diagonal conflicts prune many of them.

Q41. What is the space complexity of this implementation?

The board uses $O(N^2)$ space and recursion uses O(N) stack space. Total space is $O(N^2)$, dominated by the board.

Q42. What is the maximum recursion depth?

At most N recursive row levels are needed, so the recursion stack is O(N).

Q43. What is brute force, and how is it different?

Brute force generates candidate arrangements and tests them, often after constructing a complete arrangement. Backtracking checks constraints during construction and stops exploring a partial arrangement as soon as it becomes invalid.

Q44. Why is brute force generally less efficient here?

It may spend time generating and checking arrangements that already contain conflicts. Backtracking prunes those invalid partial arrangements early.

D. Alternatives, applications and improvements

Q45. Can N-Queens be solved iteratively?

Yes. An explicit stack can simulate the recursive search, but recursion expresses the row-by-row choices and undo operations more simply.

Q46. What is bitmask backtracking?

It represents occupied columns and diagonals as bits in integer masks. Safety checks and updates can then be performed with bitwise operations, often much faster than scanning the board.

Q47. What is constraint satisfaction?

It is a problem-solving model where variables must be assigned values while satisfying constraints. In N-Queens, each row's variable is its Queen's column, and constraints prevent column and diagonal conflicts.

Q48. How can this program be modified to find all solutions?

When row == N, save or print the current board, then return to continue searching rather than immediately returning true. The recursive function can count or collect all solutions.

Q49. How can the program be modified to accept different board sizes?

Read N at runtime and use a dynamically sized structure such as vector> instead of a compile-time fixed array.

Q50. What is one limitation of fixing the first Queen at (0,0)?

The program searches only for solutions compatible with that fixed position. It does not enumerate solutions that require the first Queen to be in another cell.

Q51. Does fixing the first Queen at (0,0) make the problem impossible?

No. The 8-Queens problem has valid solutions with a Queen at (0,0), and this program finds one of them.

Q52. Why is the first Queen fixed in this practical?

It is a condition given by the problem statement. Fixing it reduces the search because the program does not need to choose a position in the first row.

Q53. What happens if no safe column exists in a row?

The function returns false to its caller. The caller then removes its previously placed Queen and tries its next available column.

Q54. What is the role of return false at the end of solveNQueens?

It indicates that every column in the current row was tried and none led to a complete solution.

Q55. What are the applications of backtracking?

Sudoku solving, maze path search, graph coloring, crossword filling, permutation generation, scheduling, and constraint-based puzzles.

Q56. How is backtracking used in Sudoku?

It tries a number in an empty cell, checks row, column and box constraints, moves to another cell if valid, and removes the number if it leads to a dead end.

Q57. How is backtracking used in graph coloring?

It assigns a color to each vertex while ensuring adjacent vertices have different colors. If no color works for a vertex, it undoes an earlier assignment.

Q58. What are the advantages of backtracking?

It avoids exploring many invalid branches, can find a solution without enumerating every complete arrangement, and provides a systematic way to solve constraint problems.

Q59. What are the disadvantages of backtracking?

Worst-case search can be exponential or factorial, performance depends on pruning and search order, and large problems may require substantial time.

Q60. What improvements can make this implementation faster?

Use bitmasks for columns and diagonals, choose a good search order, and maintain conflict information so the program does not repeatedly scan the board.

Last-minute revision table

Concept	Recall point
N-Queens	N Queens; no shared row, column or diagonal.
Board values	1 = Queen; 0 = empty.
Fixed first Queen	(0,0); start recursion at row 1.
isSafe()	Check same column and both upper diagonals.
Base case	row == N → solution found.
Backtracking line	board[row][col] = 0; undo a failed placement.
One solution	Row-wise columns: 0, 4, 7, 5, 2, 6, 1, 3.
Time estimate	$O(N \cdot N!)$ with O(N) safety checking.
Space	$O(N^2)$ board + O(N) recursion = $O(N^2)$.
Recursion	A function calls itself.
Backtracking	Try, check, recurse, undo if needed.